

Deadhunt: Dead Heat Competition Analysis

A deep dive into horse racing data and betting optimization

Why did the data scientist break up with the horse? It had too many 'outliers'! :)

This notebook provides a complete and structured analysis of the `race_data.csv` dataset.

```
!pip install seaborn

Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
    294.9/294.9 KB 1.9 MB/s eta
0:00:00a 0:00:01
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in
./venv/lib/python3.10/site-packages (from seaborn) (3.10.9)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in
./venv/lib/python3.10/site-packages (from seaborn) (2.2.6)
Requirement already satisfied: pandas>=1.2 in
./venv/lib/python3.10/site-packages (from seaborn) (2.3.3)
Requirement already satisfied: kiwisolver>=1.3.1 in
./venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn) (1.5.0)
Requirement already satisfied: pyparsing>=3 in
./venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn) (3.3.2)
Requirement already satisfied: packaging>=20.0 in
./venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn) (26.2)
Requirement already satisfied: cycler>=0.10 in
./venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
./venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn) (4.62.1)
Requirement already satisfied: contourpy>=1.0.1 in
./venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn) (1.3.2)
Requirement already satisfied: pillow>=8 in
./venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn) (12.2.0)
Requirement already satisfied: python-dateutil>=2.7 in
./venv/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4-
>seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
./venv/lib/python3.10/site-packages (from pandas>=1.2->seaborn)
(2026.1.post1)
```

```
Requirement already satisfied: tzdata>=2022.7 in
./venv/lib/python3.10/site-packages (from pandas>=1.2->seaborn)
(2026.2)
Requirement already satisfied: six>=1.5 in
./venv/lib/python3.10/site-packages (from python-dateutil>=2.7-
>matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import norm, lognorm, shapiro

# Setting aesthetic style
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams['figure.figsize'] = (12, 6)
```

1. Data Loading & Inspection

We start by loading the 500 historical races.

```
df = pd.read_csv('race_data.csv')
horses = ['Shadowfax', 'Iron Duke', 'Morningstar', 'Red Tide',
'Gallant Fox', 'Blue Streak', 'Copper Prince', 'Last Chance']
print(f"Dataset Shape: {df.shape}")
df.head()
```

Dataset Shape: (500, 10)

	race_id	Shadowfax	Iron Duke	Morningstar	Red Tide	Gallant
Fox \						
0	0	67.5041	65.6016	70.4587	73.1363	64.2286
1	1	66.6415	66.0323	71.0060	72.6026	71.0442
2	2	67.6772	65.7881	71.0019	69.9481	70.1585
3	3	65.5535	67.2001	69.5425	71.2678	72.2864
4	4	65.3339	66.1231	69.8925	73.7591	70.4077
	Blue Streak	Copper Prince	Last Chance	winner		
0	67.2475	75.0616	75.5429	Gallant Fox		
1	76.8610	76.7373	72.3313	Iron Duke		
2	69.5850	80.5973	76.5265	Iron Duke		

3	73.9726	85.5591	74.9999	Shadowfax
4	68.9783	70.5561	81.6178	Shadowfax

2. Descriptive Statistics (Minute Details)

Let's look at every possible metric for each horse's finishing time.

```
stats_df = df[horses].describe().T
stats_df['skew'] = df[horses].skew()
stats_df['kurtosis'] = df[horses].kurtosis()
stats_df['median'] = df[horses].median()
stats_df.sort_values('mean')
```

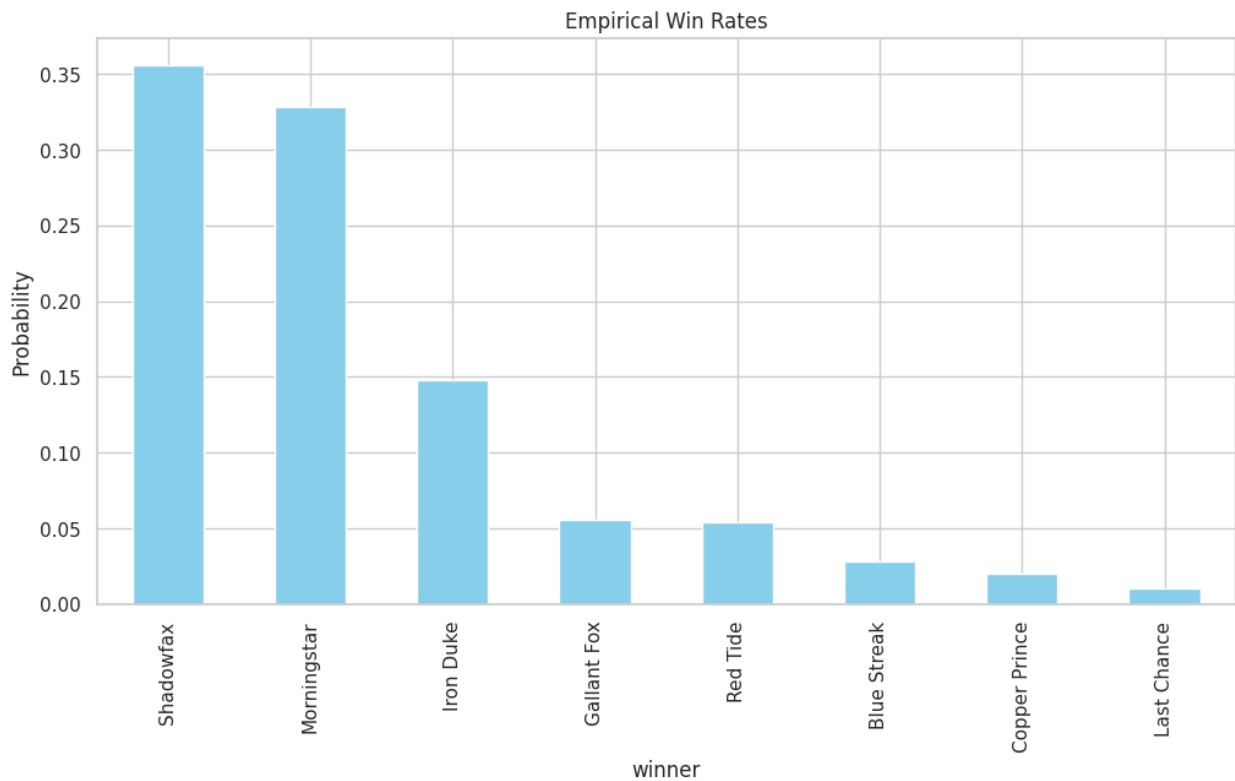
	count	mean	std	min	25%
50% \					
Shadowfax	500.0	66.664992	2.590587	59.2294	64.952925
66.52610					
Morningstar	500.0	67.324614	4.133334	57.1131	64.383600
67.44575					
Iron Duke	500.0	68.064353	2.327766	61.4301	66.520325
68.12615					
Red Tide	500.0	70.146380	3.167742	61.0767	67.953500
70.15815					
Gallant Fox	500.0	70.878176	3.501893	61.6270	68.577025
70.78600					
Blue Streak	500.0	72.334719	4.052565	59.1063	69.659000
72.26860					
Copper Prince	500.0	74.389887	4.844581	62.2044	70.910000
74.40805					
Last Chance	500.0	77.589519	6.384608	62.2300	72.766825
77.26250					
	75%	max	skew	kurtosis	median
Shadowfax	68.177025	75.4553	0.395402	0.560735	66.52610
Morningstar	70.133025	79.2530	0.134738	-0.260961	67.44575
Iron Duke	69.642000	75.0635	-0.008040	0.064933	68.12615
Red Tide	72.132800	79.9278	-0.060209	-0.019195	70.15815
Gallant Fox	73.154050	81.6355	0.142784	-0.105119	70.78600
Blue Streak	74.836975	85.3488	0.104235	0.348598	72.26860
Copper Prince	77.532300	91.5262	0.230425	0.008436	74.40805
Last Chance	81.583400	98.9646	0.376688	-0.062001	77.26250

3. Historical Win Rates

How often did each horse actually win in the 500 races?

```
win_counts = df['winner'].value_counts(normalize=True)
win_counts.plot(kind='bar', color='skyblue')
```

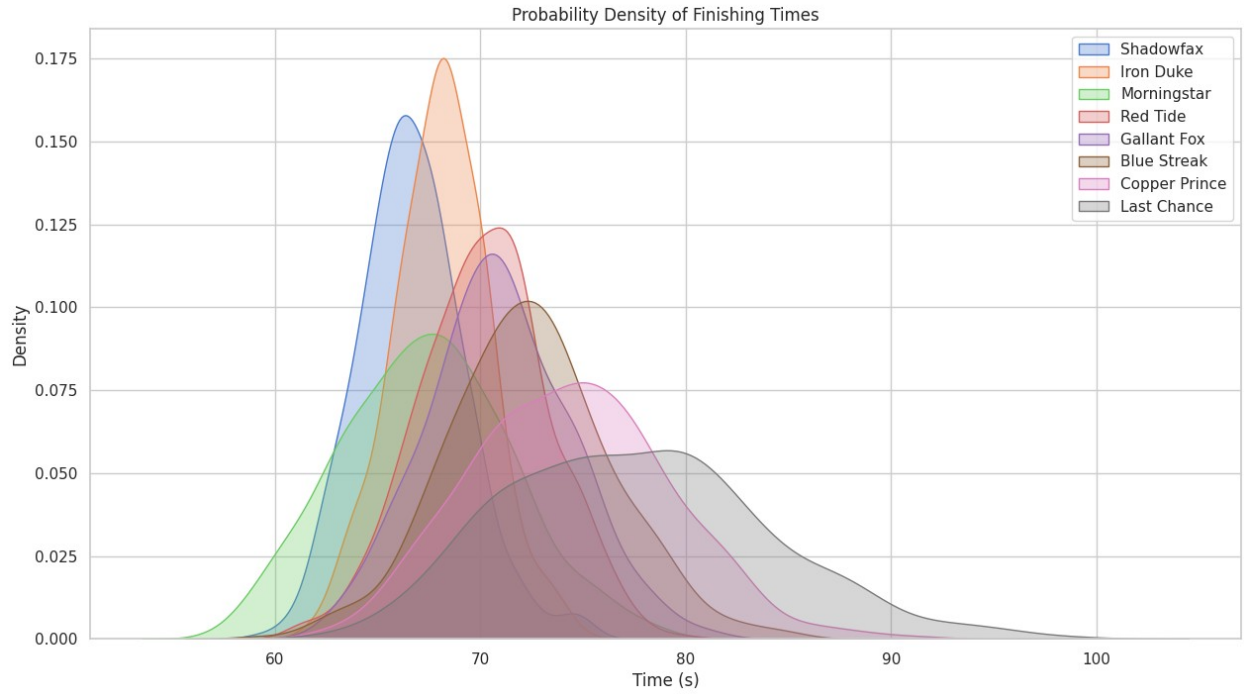
```
plt.title("Empirical Win Rates")
plt.ylabel("Probability")
plt.show()
```



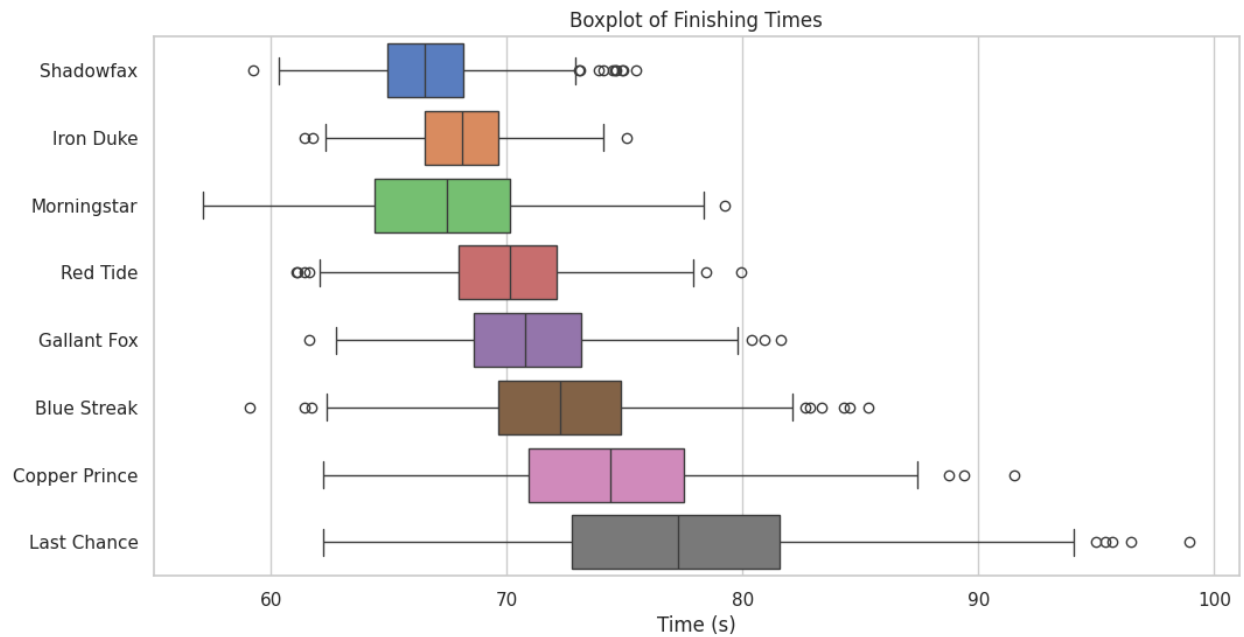
4. Visualizing Performance Distributions

Understanding the spread and overlap of finishing times.

```
plt.figure(figsize=(15, 8))
for horse in horses:
    sns.kdeplot(df[horse], label=horse, fill=True, alpha=0.3)
plt.title("Probability Density of Finishing Times")
plt.xlabel("Time (s)")
plt.legend()
plt.show()
```



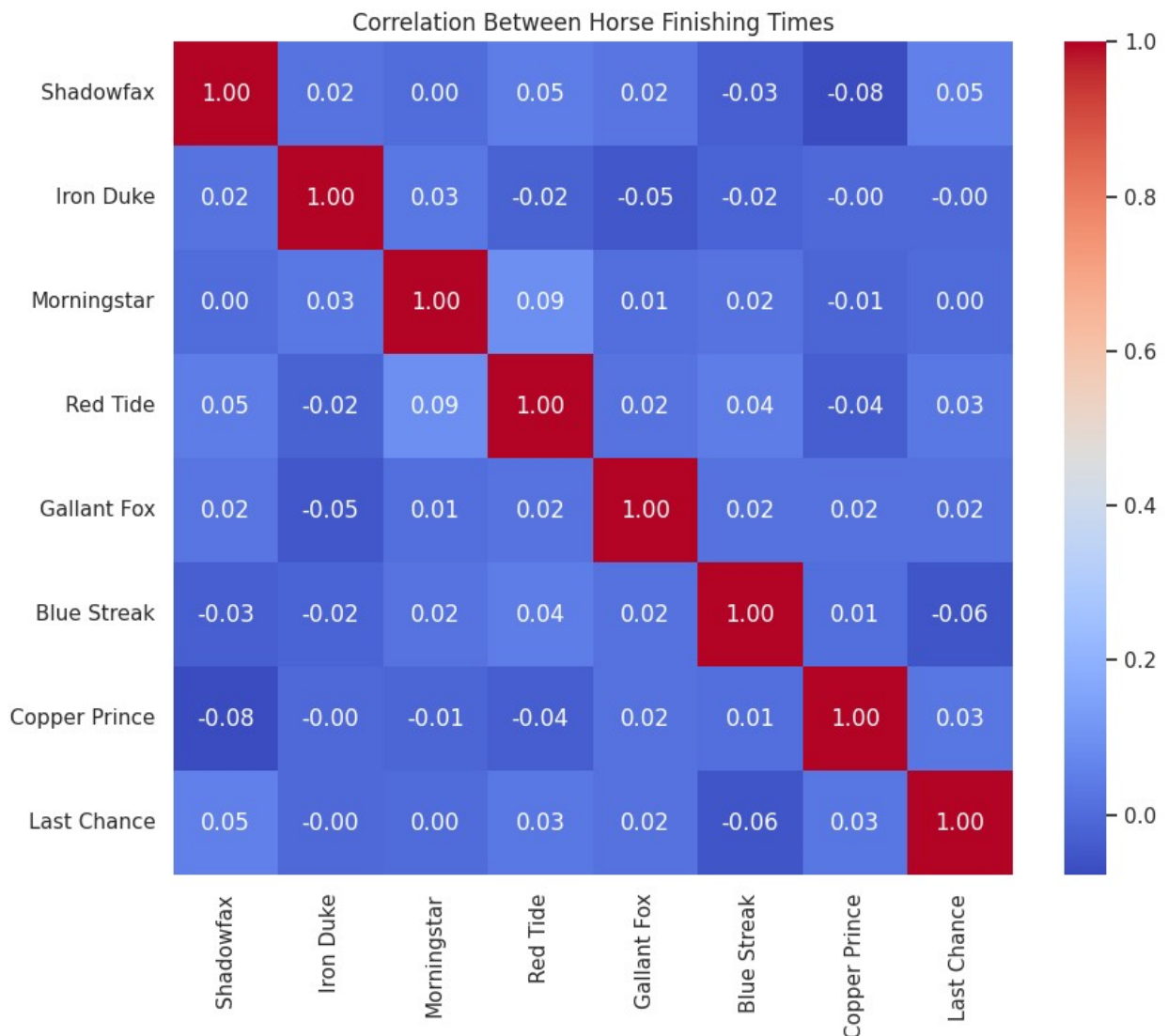
```
plt.figure(figsize=(12, 6))
sns.boxplot(data=df[horses], orient='h')
plt.title("Boxplot of Finishing Times")
plt.xlabel("Time (s)")
plt.show()
```



5. Correlation Analysis

Are horses affected by common race conditions (e.g., track state)? High correlation would suggest external factors.

```
plt.figure(figsize=(10, 8))
sns.heatmap(df[horses].corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Between Horse Finishing Times")
plt.show()
```



6. Distribution Fitting & Refined Simulation

Determining if times are Normal or Log-Normal to improve simulation accuracy.

```

best_fits = {}
for horse in horses:
    times = df[horse]
    stat, p = shapiro(times)

    shape, loc, scale = lognorm.fit(times, floc=0)
    lognorm_ll = lognorm.logpdf(times, shape, loc, scale).sum()
    norm_ll = norm.logpdf(times, times.mean(), times.std()).sum()

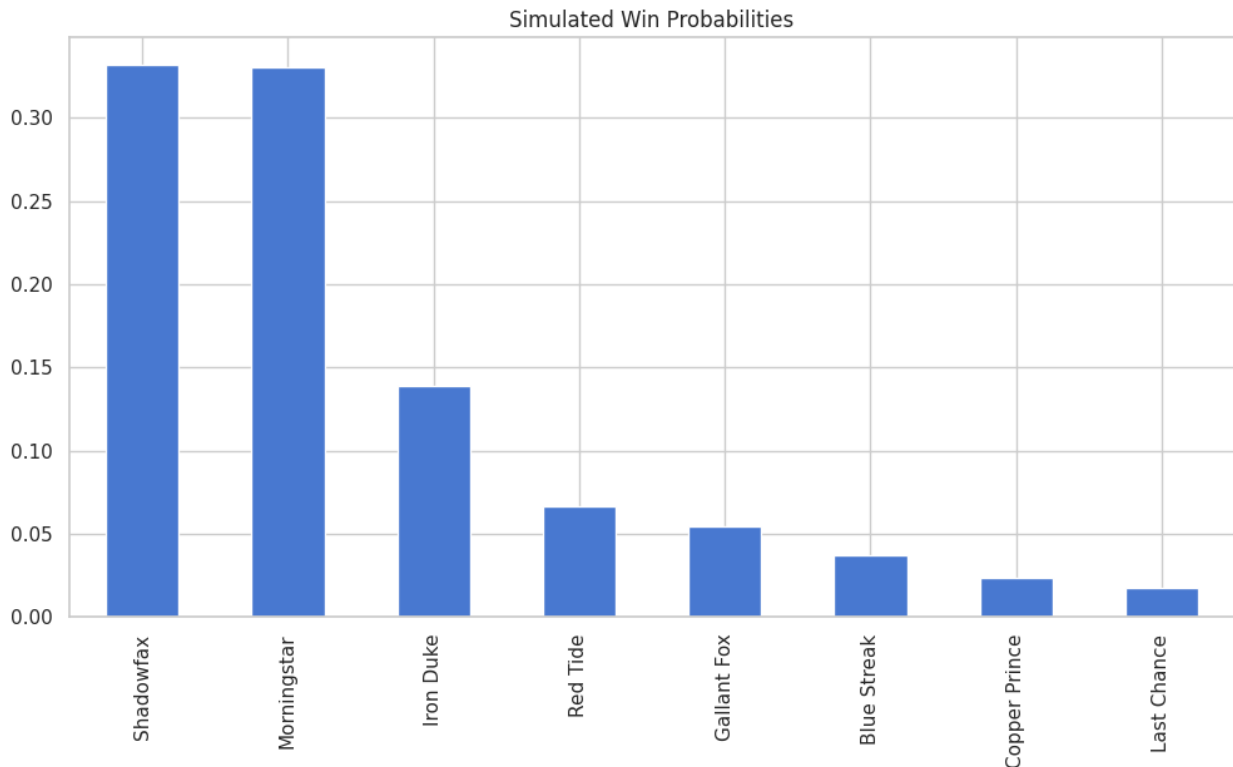
    best_fit = "Normal" if norm_ll > lognorm_ll else "Log-Normal"
    best_fits[horse] = (best_fit, shape, loc, scale, p)
    print(f"{horse:<15}: Best Fit = {best_fit:<12} (Shapiro
p={p:.4f})")

# Monte Carlo Simulation
n_sims = 200000
sim_results = np.zeros((n_sims, len(horses)))
for i, horse in enumerate(horses):
    fit_type, shape, loc, scale, _ = best_fits[horse]
    if fit_type == "Normal":
        sim_results[:, i] = np.random.normal(df[horse].mean(),
df[horse].std(), n_sims)
    else:
        sim_results[:, i] = lognorm.rvs(shape, loc, scale,
size=n_sims)

winners = np.argmax(sim_results, axis=1)
win_probs = {horses[i]: np.sum(winners == i) / n_sims for i in
range(len(horses))}
pd.Series(win_probs).sort_values(ascending=False).plot(kind='bar')
plt.title("Simulated Win Probabilities")
plt.show()

```

Shadowfax	: Best Fit = Log-Normal	(Shapiro p=0.0006)
Iron Duke	: Best Fit = Normal	(Shapiro p=0.4648)
Morningstar	: Best Fit = Log-Normal	(Shapiro p=0.1785)
Red Tide	: Best Fit = Normal	(Shapiro p=0.7733)
Gallant Fox	: Best Fit = Log-Normal	(Shapiro p=0.5087)
Blue Streak	: Best Fit = Log-Normal	(Shapiro p=0.4217)
Copper Prince	: Best Fit = Log-Normal	(Shapiro p=0.1235)
Last Chance	: Best Fit = Log-Normal	(Shapiro p=0.0006)



7. Betting Strategies & Final Decision

Comparing the market to our refined probabilities.

```
market_probs = {
    'Shadowfax': 0.08, 'Iron Duke': 0.09, 'Morningstar': 0.11, 'Red
Tide': 0.13,
    'Gallant Fox': 0.14, 'Blue Streak': 0.15, 'Copper Prince': 0.14,
    'Last Chance': 0.16
}
takeout = 0.15

evs = {h: win_probs[h] * (1-takeout) / market_probs[h] - 1 for h in
horses}

# Clubbed Stake Calculation
pos_ev_horses = [h for h in horses if evs[h] > 0]
sum_p = sum(win_probs[h] for h in pos_ev_horses)
sum_f_adj = sum(market_probs[h]/0.85 for h in pos_ev_horses)
Q = (1 - sum_p) / (1 - sum_f_adj)

strat_kelly = {h: max(0, (win_probs[h] - (market_probs[h]/0.85) * Q) *
10000) if h in pos_ev_horses else 0 for h in horses}
total_kelly = sum(strat_kelly.values())
strat_norm_kelly = {h: (strat_kelly[h] / total_kelly * 10000) if
total_kelly > 0 else 0 for h in horses}
```



```
strat_prob = {h: (win_probs[h] / sum_p * 10000) if h in pos_ev_horses
else 0 for h in horses}

final_stakes = {h: int(0.7 * strat_norm_kelly[h] + 0.3 *
strat_prob[h]) for h in horses}
final_stakes[max(evs, key=evs.get)] += (10000 -
sum(final_stakes.values()))

print("Final Recommendation:")
for h, v in final_stakes.items():
    if v > 0:
        print(f"{h}: £{v}")
```

Final Recommendation:

Shadowfax: £4269

Iron Duke: £1591

Morningstar: £4140